LAB 3 TỔNG HỢP

Computer Store Management System (Flutter + SQLite)

Introduction

Computer stores need an efficient mobile application to manage products, brands, and product categories. Besides performing basic CRUD operations, the application should support searching, filtering, sorting, and generating business statistics to help store owners monitor inventory and sales performance.

In this assignment, you are required to develop a **Flutter mobile application** using **SQLite** as the local database. The application must allow users to manage brands, categories, and computer products through an intuitive interface built with Flutter Material Design components.

The application will demonstrate your understanding of Flutter widgets, navigation, SQLite database operations, SQL queries, form validation, and local data management.

General Requirements

A resource folder is provided containing:

- Sample SQLite database (optional)
- Sample JSON data
- Application icons
- UI mockups
- Expected screenshots

Create a Flutter project named
`ComputerStoreSys`

Development Technologies

- Flutter SDK (Stable)
- Dart
- SQLite (sqflite)
- path
- path_provider
- Provider (Optional)

Development Tools

- Android Studio
- Visual Studio Code
- Android Emulator
- Chrome Browser (Optional)
- DB Browser for SQLite (Optional)

Database

Database Name

`ComputerStoreDB.db`

Tables

The application consists of **three tables**.

Brands

Field	Type	Description
id	INTEGER	Primary Key AUTOINCREMENT
brand_name	TEXT	Brand Name
country	TEXT	Country
founded_year	INTEGER	Founded Year

Categories

Field	Type	Description
id	INTEGER	Primary Key AUTOINCREMENT
category_name	TEXT	Category Name

Allowed Category Values

- Laptop
- Desktop
- Component

Phucpt10@fpt.edu.vn

Computers

Field	Type	Description
id	INTEGER	Primary Key AUTOINCREMENT
product_name	TEXT	Product Name
brand_id	INTEGER	Foreign Key
category_id	INTEGER	Foreign Key
cpu	TEXT	CPU
ram	INTEGER	RAM (GB)
storage	INTEGER	Storage (GB)
price	REAL	Price
stock	INTEGER	Stock
sold_quantity	INTEGER	Sold Quantity
is_featured	INTEGER	0=False, 1=True

Relationships

```
Brands (1)
       |
       |------< Computers >------|
                                 |
                          Categories (1)
```

Each computer belongs to exactly one Brand and one Category.

Navigation Structure

The application must use **BottomNavigationBar**.
The application contains **6 tabs**.

Tab	Description
Store	Display computer products
Add Computer	Add new computer
Brands	Brand management
Categories	Category management
Statistics	Business statistics
Featured	Featured products

Home Screen Layout

The **Store** tab is the **Home Screen** of the application and is displayed by default when the application starts.

The Home Screen must contain the following components in order:

1. AppBar
2. Promotional Banner
3. Search Bar
4. Filter Button
5. Sort Button
6. Computer Product List
7. BottomNavigationBar

Requirements

- The Store tab is the default page when the application starts.
- The promotional banner must be displayed before the product list.
- The Search Bar must be displayed below the banner.
- The Filter and Sort controls must be displayed above the product list.
- The product list must support scrolling independently.

Phucpt10@fpt.edu.vn

- BottomNavigationBar must always remain visible for switching between tabs.

Task 1. Project Initialization

Description

Initialize a Flutter project and configure SQLite database.

Requirements

Create a Flutter project named
`ComputerStoreSys`
Configure dependencies

```
sqflite
path
path_provider
provider (optional)
```

Create SQLite database
`ComputerStoreDB.db`

Create the following tables

- brands
- categories
- computers

Insert sample data
Minimum requirements

- 5 Brands
- 3 Categories
- 15 Computers

The application must run successfully on

- Android Emulator (Required)

or

- Chrome Browser (Optional)

Task 2. Brand Management

Description

Develop a complete Brand Management module.
The module allows users to create, update, delete and search brands.

2.1 Display Brand List

Display all brands stored in SQLite.
Each Brand should display

- Brand Name
- Country
- Founded Year
- Number of Products

Example

```
Country : Taiwan

Founded : 1989

Products : 12
```

Requirements

Retrieve data from SQLite.
Display data using

- ListView.builder()

Phucpt10@fpt.edu.vn

or

- ListView.separated()

Each Brand should be displayed using

- Card

or

- ListTile

The number of products must be calculated using SQL JOIN.
Example SQL

```
SELECT

b.id,

b.brand_name,

b.country,

b.founded_year,

COUNT(c.id) AS totalProducts

FROM brands b

LEFT JOIN computers c

ON b.id=c.brand_id

GROUP BY b.id
```

2.2 Add Brand

Allow users to create a new Brand.
Input fields

- Brand Name
- Country
- Founded Year

Validation

- Brand Name cannot be empty.
- Country cannot be empty.
- Founded Year must be greater than 1900.
- Duplicate Brand Name is not allowed.

After Save

- Insert into SQLite.
- Display Snackbar

```
Brand added successfully.
```

Automatically refresh Brand List.

2.3 Edit Brand

Allow users to update

- Brand Name
- Country
- Founded Year

Requirements

- Pre-fill all information.
- Update SQLite.
- Refresh Brand List.

2.4 Delete Brand

Display confirmation dialog

```
Are you sure you want to delete this brand?
```

Before deleting

Check whether the Brand is being used by any Computer.

SQL

```
SELECT COUNT(*)

FROM computers

WHERE brand_id=?
```

If count > 0

Display

```
This brand cannot be deleted because it is currently used by one or
more products.
```

Otherwise

Delete successfully.

2.5 Search Brand

Search conditions

- Brand Name
- Country

Requirements

- Ignore uppercase/lowercase.
- Search in real-time.

Example SQL

```
SELECT *

FROM brands

WHERE

brand_name LIKE ?

OR

country LIKE ?
```

2.6 Display Promotional Banner

Description

Design a promotional banner displayed at the top of the **Store Screen (Home Screen)** to improve the application's user interface and user experience.

The banner image must be loaded from the **assets/images** folder.

Requirements

- Add a banner image at the top of the Store Screen.
- The banner must span the full width of the screen.
- The banner should have rounded corners.
- Maintain the image aspect ratio.
- The banner should be displayed before the product list.
- Store the image inside the project folder:

Task 3. Category Management

Description

Develop a Category Management module.
The module allows users to manage all product categories.

3.1 Display Category List

Display

- Category Name
- Total Products

Example

```
Laptop

Products : 8
```

The total number of products must be calculated using SQL JOIN.
Example SQL

```
SELECT

ca.id,

ca.category_name,

COUNT(c.id)

FROM categories ca

LEFT JOIN computers c

ON ca.id=c.category_id

GROUP BY ca.id
```

3.2 Add Category

Input

- Category Name

Validation

- Category Name is required.
- Category Name must be unique.

After Save
Display

```
Category added successfully.
```

3.3 Edit Category

Allow updating

- Category Name

Requirements

- Update SQLite.
- Refresh Category List.

3.4 Delete Category

Display

```
Are you sure you want to delete this category?
```

Before deleting
Check whether the Category is currently used.
SQL

Phucpt10@fpt.edu.vn

```
SELECT COUNT(*)

FROM computers

WHERE category_id=?
```

If the category is used
Display
`This category cannot be deleted because one or more products belong to this category.`
Otherwise
Delete successfully.

3.5 Search Category

Search by

- Category Name

Requirements

- Real-time search.
- Ignore uppercase/lowercase.

Example SQL

```
SELECT *

FROM categories

WHERE category_name LIKE ?
```

Task 4. Computer Management

Description

Develop a **Computer Management** module that allows users to manage all computer products stored in the SQLite database.
Each computer belongs to **one Brand** and **one Category**.
The module must support:

- Display product list
- View product details
- Add a new computer
- Update computer information
- Delete a computer
- Store all data in SQLite

4.1 Display Computer List

Description

Display all computer products stored in SQLite.
The application **must retrieve data using SQL JOIN** to display the corresponding Brand and Category names.

Requirements

Display the following information for each product.

- Product Name
- Brand Name
- Category Name
- CPU
- RAM
- Storage
- Price
- Stock
- Sold Quantity
- Featured Icon (⭐)

Example
`ROG Strix G16`

Phucpt10@fpt.edu.vn

```
Brand      : ASUS
Category   : Laptop
CPU         : Intel Core i7
RAM         : 16 GB
Storage     : 1000 GB
Price       : $1,250
Stock       : 15
Sold        : 32
```
⭐ Featured

Display products using

- Card
- ListTile
- Custom Widget

The list must be built using
`ListView.builder()`
or
`ListView.separated()`

Required SQL

```
SELECT

c.id,
c.product_name,

b.brand_name,

ca.category_name,

c.cpu,
c.ram,
c.storage,
c.price,
c.stock,
c.sold_quantity,
c.is_featured

FROM computers c

INNER JOIN brands b
ON c.brand_id = b.id

INNER JOIN categories ca
ON c.category_id = ca.id

ORDER BY c.product_name ASC;
```

Additional Requirements

Implement Pull-to-Refresh using
`RefreshIndicator`
Clicking a product must navigate to
`Computer Detail Screen`
using
`Navigator.push()`

4.2 Add Computer

Description

Allow users to create a new computer product.

Phucpt10@fpt.edu.vn

Input Fields

Field	Widget
Product Name	TextFormField
Brand	DropdownButtonFormField
Category	DropdownButtonFormField
CPU	TextFormField
RAM	TextFormField
Storage	TextFormField
Price	TextFormField
Stock	TextFormField
Sold Quantity	TextFormField
Featured	Switch

Requirements

Brand Dropdown
Load data from
`brands`
Category Dropdown
Load data from
`categories`
Example
`Brand`

▼ `ASUS`
`Acer`
`Dell`
`HP`

Validation

All fields are required.
Price
`Price > 0`
RAM
`4 GB → 128 GB`
Storage
`128 GB → 4096 GB`
Stock
`Stock ≥ 0`
Sold Quantity
`Sold Quantity ≥ 0`
Product Name
Must be unique.

After Save

- Insert data into SQLite.
- Display Snackbar

`Computer added successfully.`
Automatically return to
`Store Screen`

Required SQL

```
INSERT INTO computers(

product_name,

brand_id,

category_id,
```


Phucpt10@fpt.edu.vn

```
cpu,

ram,

storage,

price,

stock,

sold_quantity,

is_featured

)

VALUES(?,?,?,?,?,?,?,?,?,?)
```

4.3 Edit Computer

Description

Allow users to modify existing computer information.

Allow updating

- Product Name
- Brand
- Category
- CPU
- RAM
- Storage
- Price
- Stock
- Sold Quantity
- Featured

Requirements

- Pre-fill all existing values.
- Validate all input.
- Update SQLite.
- Refresh Store Screen.

Required SQL

```
UPDATE computers

SET

product_name=?,

brand_id=?,

category_id=?,

cpu=?,

ram=?,

storage=?,

price=?,
```

```
stock=?,

sold_quantity=?,

is_featured=?

WHERE id=?
```

4.4 Delete Computer

Description

Delete a computer product.

Display confirmation dialog
`Are you sure you want to delete this product?`
Buttons
`Cancel`

`Delete`

If confirmed
Delete the record.
Display
`Computer deleted successfully.`
Return to Store Screen.

Required SQL

```
DELETE

FROM computers

WHERE id=?
```

4.5 Product Detail

Description

Display all product information.
Retrieve data using SQL JOIN.

Display

- Product Name
- Brand
- Category
- CPU
- RAM
- Storage
- Price
- Stock
- Sold Quantity
- Featured

Business Calculations

The following values are **not stored in the database**.
Students must calculate them inside Flutter.

Revenue
`Revenue = Price × Sold Quantity`

Inventory Value
`Inventory Value = Price × Stock`

Phucpt10@fpt.edu.vn

Total Units

```
Total Units = Stock + Sold Quantity
```

Example

```
Revenue

1250 × 32

=

40000
```

Required SQL

```
SELECT

c.*,

b.brand_name,

ca.category_name

FROM computers c

INNER JOIN brands b

ON c.brand_id=b.id

INNER JOIN categories ca

ON c.category_id=ca.id

WHERE c.id=?
```

4.6 Product Validation Rules

Field	Validation
Product Name	Required, Unique
Brand	Required
Category	Required
CPU	Required
RAM	4–128
Storage	128–4096
Price	> 0
Stock	≥ 0
Sold Quantity	≥ 0

4.7 SQLite Operations Summary

Students must implement the following database methods.

```
insertComputer()

updateComputer()

deleteComputer()

getComputerById()

getAllComputers()
```


Phucpt10@fpt.edu.vn

4.8 Learning Outcomes

After completing **Task 4**, students should be able to

- Create relational data using **Foreign Keys**.
- Load data into **DropdownButtonFormField** from SQLite.
- Implement complete CRUD operations.
- Navigate between multiple Flutter screens.
- Display related data using **INNER JOIN**.
- Apply form validation.
- Use `SnackBar`, `AlertDialog`, and `Navigator`.
- Calculate business values in Flutter without storing redundant data.
- Organize the application following a layered architecture (Model – Database – Service – UI).

Task 5. Search, Filter and Sort

Description

The **Store** screen must support searching, filtering, and sorting computer products. All operations must retrieve data from **SQLite**. Students are encouraged to build dynamic SQL queries based on user-selected conditions.

The search and filter results must display complete product information, including **Brand Name** and **Category Name**, by using **SQL JOIN**.

5.1 Search Computers

Description

Allow users to search computer products in real time.

Supported Search Fields

- Product Name
- Brand Name
- CPU

Example

```
Search: ASUS
```

Expected Result

```
ROG Strix G16
Vivobook S15
TUF Gaming A15
```

Requirements

- Search must be performed in real time.
- Ignore uppercase/lowercase.
- Display matching records immediately.
- Search should support partial keywords.

Required SQL

```
SELECT

c.id,
c.product_name,
b.brand_name,
ca.category_name,
c.cpu,
c.ram,
c.storage,
c.price,
c.stock,
c.sold_quantity,
c.is_featured

FROM computers c

INNER JOIN brands b
ON c.brand_id=b.id
```


Phucpt10@fpt.edu.vn

```
INNER JOIN categories ca
ON c.category_id=ca.id

WHERE

c.product_name LIKE ?

OR b.brand_name LIKE ?

OR c.cpu LIKE ?

ORDER BY c.product_name;
```

5.2 Filter Computers

Description

Users can filter products using one or multiple conditions.

Filter by Brand

Display all available brands in a Dropdown.
Example
`Brand`

▼ ASUS

Filter by Category

Supported Categories

- Laptop
- Desktop
- Component

Filter by Featured

Switch
`Featured Only`
When enabled
`WHERE is_featured=1`

Filter by Stock Status

Supported values

Status	Condition
In Stock	stock > 0
Low Stock	stock < 5
Out of Stock	stock = 0

Filter by RAM

Supported options

- 8 GB
- 16 GB
- 32 GB or above

Example SQL
`WHERE ram>=32`

Filter by Storage

Supported options

- 256 GB
- 512 GB
- 1 TB
- 2 TB+

Filter by Price

Supported options

- Less than 500
- 500 – 1000
- Greater than 1000

Example SQL

```
WHERE price BETWEEN 500 AND 1000
```

5.3 Multiple Filters

The application must support combining multiple filters.

Example

```
Brand

ASUS

Category

Laptop

RAM

16 GB

Price

500-1000
```

Expected SQL

```
SELECT *

FROM computers c

INNER JOIN brands b

ON c.brand_id=b.id

INNER JOIN categories ca

ON c.category_id=ca.id

WHERE

b.id=?

AND

ca.id=?

AND

ram=16

AND

price BETWEEN 500 AND 1000;
```

5.4 Sort Computers

Description

Allow users to sort computer products.

Supported Sort Options

Price

- Ascending
- Descending

SQL

```
ORDER BY price ASC
ORDER BY price DESC
```

Sold Quantity

- Ascending
- Descending

Product Name

- A → Z
- Z → A

Brand Name

- A → Z
- Z → A

Stock

- Low → High
- High → Low

5.5 Reset Filters

Provide one button

```
Reset Filters
```

The button should clear

- Search keyword
- Brand
- Category
- Featured
- RAM
- Storage
- Price
- Stock Status
- Sort option

and reload all products.

5.6 Advanced Search

Allow users to search using multiple conditions simultaneously.

Example

```
Keyword

Gaming

Brand

ASUS

Category

Laptop

Price
```

```
>1000

Featured

Yes
```

Expected SQL

```
SELECT

c.*,

b.brand_name,

ca.category_name

FROM computers c

INNER JOIN brands b

ON c.brand_id=b.id

INNER JOIN categories ca

ON c.category_id=ca.id

WHERE

c.product_name LIKE '%Gaming%'

AND

b.brand_name='ASUS'

AND

ca.category_name='Laptop'

AND

price>1000

AND

is_featured=1;
```

5.7 Search Result

Each search result should display

- Product Name
- Brand
- Category
- CPU
- RAM
- Storage
- Price
- Stock
- Sold Quantity
- Featured

5.8 SQLite Methods

Students must implement

```
searchComputers()

filterComputers()

sortComputers()

resetFilters()
```

5.9 Learning Outcomes

After completing Task 5, students should be able to

- Perform real-time searching using SQLite.
- Build dynamic SQL statements.
- Use `LIKE`, `WHERE`, `BETWEEN`, `ORDER BY`, and `AND`.
- Retrieve data using **INNER JOIN**.
- Combine multiple search conditions.
- Develop user-friendly search and filter interfaces.

Task 6. Statistics Screen

Description

Develop a **Statistics** screen to summarize business information. All statistical values **must be calculated directly from SQLite** using SQL aggregate functions. The application **must not hard-code** any statistical values.

6.1 General Statistics

Display the following information:

- Total Products
- Total Revenue
- Total Sold Quantity
- Average Product Price
- Total Inventory Value

Total Products

```
SELECT COUNT(*)

FROM computers;
```

Total Revenue

```
SELECT

SUM(price*sold_quantity)

FROM computers;
```

Total Sold Quantity

```
SELECT

SUM(sold_quantity)

FROM computers;
```

Average Price

```
SELECT

AVG(price)

FROM computers;
```

Inventory Value

```
SELECT

SUM(price*stock)

FROM computers;
```

6.2 Statistics by Brand

Display

- Brand Name
- Number of Products
- Total Sold Quantity
- Total Revenue
- Inventory Value

Required SQL

```
SELECT

b.brand_name,

COUNT(c.id) AS TotalProducts,

SUM(c.sold_quantity) AS TotalSold,

SUM(c.price*c.sold_quantity) AS Revenue,

SUM(c.price*c.stock) AS InventoryValue

FROM brands b

LEFT JOIN computers c

ON b.id=c.brand_id

GROUP BY b.id

ORDER BY Revenue DESC;
```

6.3 Statistics by Category

Display

- Category Name
- Total Products
- Total Revenue
- Total Sold

Required SQL

```
SELECT

ca.category_name,

COUNT(c.id),

SUM(c.price*c.sold_quantity),

SUM(c.sold_quantity)

FROM categories ca

LEFT JOIN computers c
```

```
ON ca.id=c.category_id

GROUP BY ca.id;
```

6.4 Best Selling Product

Display the product with the highest sold quantity.

```
SELECT

product_name,

sold_quantity

FROM computers

ORDER BY sold_quantity DESC

LIMIT 1;
```

6.5 Most Expensive Product

```
SELECT

product_name,

price

FROM computers

ORDER BY price DESC

LIMIT 1;
```

6.6 Low Stock Products

Display all products whose stock is less than 5.

```
SELECT

c.product_name,

b.brand_name,

c.stock

FROM computers c

INNER JOIN brands b

ON c.brand_id=b.id

WHERE stock<5;
```

6.7 Top Brand

Display the brand with the highest total sold quantity.

```
SELECT

b.brand_name,

SUM(c.sold_quantity) AS TotalSold

FROM brands b
```

```
INNER JOIN computers c

ON b.id=c.brand_id

GROUP BY b.id

ORDER BY TotalSold DESC

LIMIT 1;
```

6.8 Featured Statistics

Display

- Total Featured Products
- Total Featured Revenue
- Total Featured Sold Quantity

```
SELECT

COUNT(*),

SUM(price*sold_quantity),

SUM(sold_quantity)

FROM computers

WHERE is_featured=1;
```

6.9 Required SQLite Methods

Students must implement

```
getStatistics()

getBrandStatistics()

getCategoryStatistics()

getBestSellingProduct()

getLowStockProducts()

getTopBrand()
```


Phucpt10@fpt.edu.vn

Giai đoạn 1. Hiểu đề và lập kế hoạch phát triển

Task A1. Phân tích đề bài

Mục tiêu

Hiểu toàn bộ yêu cầu của bài tập trước khi viết code.

Prompt 1

```
Read the following assignment carefully.

Act as a senior Flutter instructor.

Summarize all functional requirements.

Separate them into UI requirements, SQLite requirements, CRUD requirements,
Search requirements, Statistics requirements, and Navigation requirements.

Explain each requirement in simple language.
```

Prompt 2

```
From this assignment, identify every Flutter screen that needs to be
developed.

Describe the purpose of each screen and the widgets that should appear on it.
```

Prompt 3

```
Create a development roadmap for this Flutter project.

Arrange all tasks from easiest to hardest.

Explain why this order is recommended.
```

Kết quả mong đợi

Sinh viên tạo được

- Mindmap
- Development Plan
- Screen List

Task A2. Thiết kế cơ sở dữ liệu

Prompt

```
Analyze the database described in the assignment.

Draw the relationships between Brands, Categories and Computers.

Explain Primary Key and Foreign Key.

Generate SQL CREATE TABLE statements for SQLite.
```

Prompt

```
Explain how INNER JOIN and LEFT JOIN work using the Computer Store database.
```


Phucpt10@fpt.edu.vn

Prompt

```
Generate sample data for

Brands

Categories

Computers

using SQLite INSERT statements.
```

Kết quả

Sinh viên phải hiểu

- PK
- FK
- JOIN

Giai đoạn 2. Chuẩn bị môi trường Flutter

Task B1. Cài đặt Flutter

Prompt

```
Explain how to install Flutter SDK on Windows.

Explain every step.

How do I verify Flutter installation?
```

Prompt

```
Explain flutter doctor output.

What should be green before starting development?
```

Task B2. Android Studio

Prompt

```
Explain how to install Android Studio for Flutter development.

Include Android Emulator configuration.
```

Prompt

```
How do I create a Pixel Android Emulator?

Explain recommended settings.
```


Phucpt10@fpt.edu.vn

Task B3. SQLite Packages

Prompt

```
What packages are required to use SQLite in Flutter?

Explain the purpose of

sqflite

path

path_provider
```

Prompt

```
Generate the pubspec.yaml dependencies required for this project.
```

Task B4. Project Structure

Prompt

```
Generate a clean Flutter project architecture for a SQLite application using

models

screens

database

services

widgets

Explain the responsibility of every folder.
```

Giai đoạn 3. Học các kiến thức Flutter

Task C1. Flutter Widgets

Prompt

```
Explain the following Flutter widgets with examples.

Scaffold

AppBar

BottomNavigationBar

Card

ListTile
```


Phucpt10@fpt.edu.vn

```
Column

Row

Expanded

Container
```

Task C2. ListView

Prompt

```
Explain ListView.builder()

When should it be used?

Show a simple example.
```

Task C3. Navigation

Prompt

```
Explain Navigator.push()

Navigator.pop()

with complete examples.
```

Task C4. Form

Prompt

```
Explain

Form

GlobalKey

TextFormField

TextEditingController

Validator

with one complete example.
```

Task C5. Dropdown

Prompt

```
Explain DropdownButtonFormField.

How do I load SQLite data into DropdownButton?
```


Phucpt10@fpt.edu.vn

Task C6. Switch

Prompt

```
Explain Switch widget.

Show how to store true/false into SQLite as integer.
```

Task C7. Snackbar

Prompt

```
Explain Snackbar.

Show success and error examples.
```

Task C8. AlertDialog

Prompt

```
Explain AlertDialog.

Show confirmation dialog before deleting SQLite data.
```

Giai đoạn 4. Học SQLite

Task D1. DatabaseHelper

Prompt

```
Explain DatabaseHelper singleton in Flutter.

Generate a standard DatabaseHelper class.
```

Task D2. CRUD

Prompt

```
Generate standard CRUD methods for SQLite in Flutter.

Insert

Update

Delete

GetById

GetAll
```


Phucpt10@fpt.edu.vn

Task D3. Models

Prompt

```
Generate Flutter Model classes for

Brand

Category

Computer

including

toMap()

fromMap()

copyWith()
```

Task D4. SQL

Prompt

```
Explain

SELECT

INSERT

UPDATE

DELETE

with Flutter SQLite examples.
```

Task D5. JOIN

Prompt

```
Explain INNER JOIN using the Computer Store database.

Generate Flutter SQLite query examples.
```

Prompt

```
Explain LEFT JOIN.

When should it be used?
```

Task D6. Aggregate Functions

Prompt

```
Explain
```

```
COUNT

SUM

AVG

MAX

MIN

GROUP BY

HAVING

using SQLite.
```

Giai đoạn 5. Thực hiện từng Task của đề

Ví dụ Task 2 Brand Management

Bước 1

Hiểu yêu cầu

Prompt

```
Explain the Brand Management module in this assignment.

List all subtasks.

Explain the expected UI.
```

Bước 2

Thiết kế UI

Prompt

```
Generate a Flutter UI for Brand List using ListView.builder and Card.
```

Bước 3

Thiết kế Database

Prompt

```
Generate SQLite CREATE TABLE statement for Brands.
```

Bước 4

Model

Prompt

```
Generate Brand model class with toMap() and fromMap().
```

Bước 5

CRUD

Prompt

```
Generate Brand CRUD methods using sqflite.
```

Bước 6

Hiển thị dữ liệu

Prompt

```
Show how to display SQLite Brand data using FutureBuilder.
```

Bước 7

Thêm dữ liệu

Prompt

```
Generate Add Brand screen with validation.
```

Bước 8

Sửa dữ liệu

Prompt

```
Generate Edit Brand screen with pre-filled data.
```

Bước 9

Xóa dữ liệu

Prompt

```
Generate Delete Brand function with AlertDialog confirmation.
```

Bước 10

Search

Prompt

```
Generate real-time Brand search using SQLite LIKE.
```

Bước 11

Review

Prompt

```
Review my Brand Management code.

Find bugs.

Suggest improvements following Flutter best practices.
```

Giai đoạn 6. AI Code Review (Bắt buộc sau mỗi Task)

Sau khi hoàn thành mỗi Task, sinh viên phải sử dụng AI để tự đánh giá trước khi nộp.

Prompt 1 – Code Review

```
Act as a senior Flutter reviewer.

Review my code for Task X.

Identify bugs, code smells, and Flutter best practice violations.

Suggest improvements without changing the functionality.
```

Prompt 2 – Refactoring

```
Refactor my code to improve readability, maintainability, and performance.

Explain every change you make.
```

Prompt 3 – Testing

```
Generate a checklist to manually test all functions of Task X, including
normal cases, boundary cases, and invalid input cases.
```


Phucpt10@fpt.edu.vn